

❖ 堆（Heap）及其操作

➤ 堆的意义与定义

【定义】“**优先队列**”（Priority Queue）是特殊的“**队列**”，从堆中取出元素的顺序是依照元素的**优先权（关键字）**大小，而不是元素进入队列的先后顺序。采用完全二叉树存储的优先队列称为**堆（Heap）**。

类型名称：**最大堆（MaxHeap）**

数据对象集：一个有 $N > 0$ 个元素的最大堆H是一棵**完全二叉树**，每个结点上的元素值**不小于**其子结点元素的值。

操作集：对于任意最多有MaxSize个元素的最大堆 $H \in \text{MaxHeap}$ ，元素 $\text{item} \in \text{ElementType}$ ，主要操作有：

- **MaxHeap Create(int MaxSize)**：创建一个空的**最大堆**。
- **Boolean IsFull(MaxHeap H)**：判断**最大堆H**是否已**满**。
- **Insert(MaxHeap H, ElementType item)**：将元素**item**插入**最大堆H**。
- **Boolean IsEmpty(MaxHeap H)**：判断**最大堆H**是否为**空**。
- **ElementType DeleteMax(MaxHeap H)**：返回**H**中**最大元素（高优先级）**。

。

➤ 若采用数组或链表实现优先队列

✎ 数组：

插入 — 元素总是插入尾部 $\sim \Theta(1)$
删除 — 查找最大（或最小）关键字 $\sim \Theta(n)$
 从数组中删去需要移动元素 $\sim O(n)$

✎ 链表：

插入 — 元素总是插入链表的头部 $\sim \Theta(1)$
删除 — 查找最大（或最小）关键字 $\sim \Theta(n)$
 删去结点 $\sim \Theta(1)$

✎ 有序数组：

插入 — 找到合适的位置 $\sim O(n)$ 或 $O(\log_2 n)$
 移动元素并插入 $\sim O(n)$
删除 — 删去最后一个元素 $\sim \Theta(1)$

✎ 有序链表：

插入 — 找到合适的位置 $\sim O(n)$
 插入元素 $\sim \Theta(1)$
删除 — 删除首元素或最后元素 $\sim \Theta(1)$

✎ 采用二叉搜索树如何？

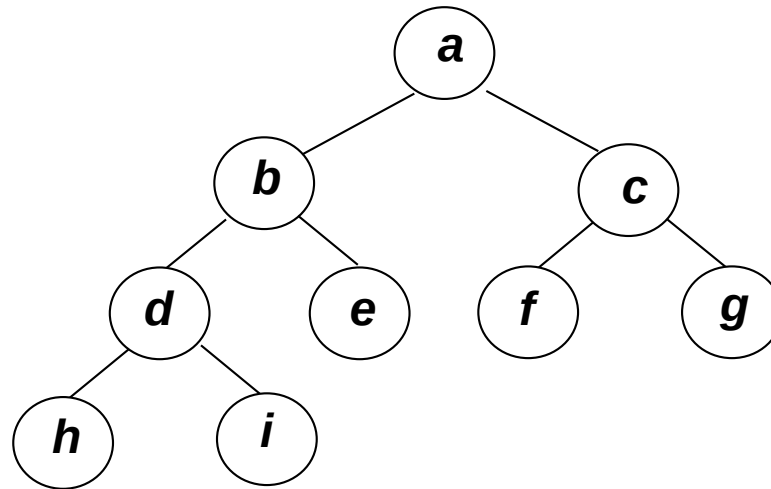
现在你开始懂 我了☺

有更好的想法了？

这，



➤ 采用完全二叉树表示优先队列



➤ “大顶堆”： $a \geq b \geq d \geq e$

➤ “小顶堆”： $a \leq c \leq f$

	a	b	c	d	e	f	g	h	i		
0	1	2	3	4	5	6	7	8	9	10	11

➤ 堆的两个特性

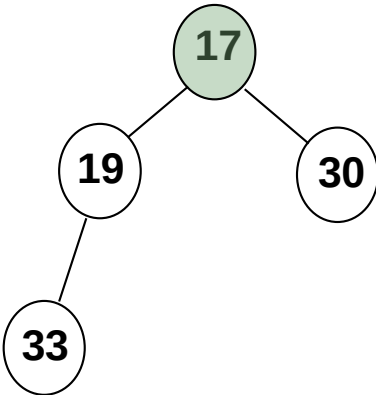
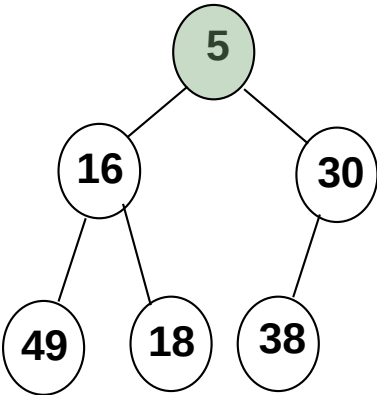
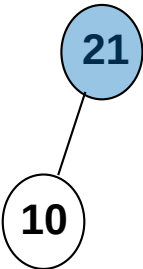
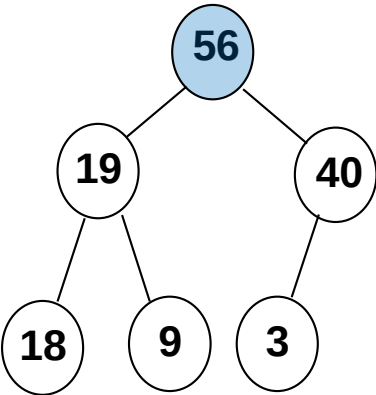
☞ **结构性**：用数组表示的完全二叉树；

☞ **有序性**：根结点到任一结点的关键字序列保持**非递增**

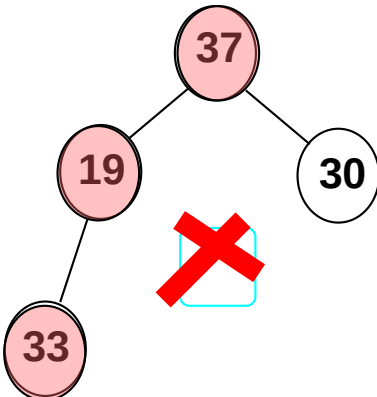
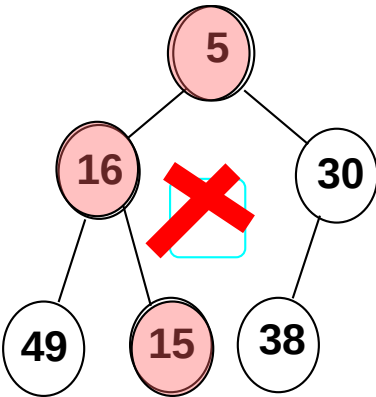
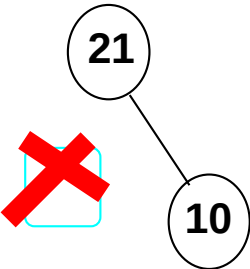
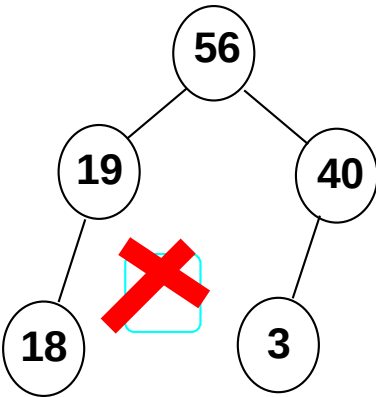
（称“**最大堆(MaxHeap)**”，也称“**大顶堆**”）或者**非递减**

（称“**最小堆(MinHeap)**”，也称“**小顶堆**”）。

【例】 最大堆和最小堆



【例】 不是堆



❖ 最大堆的操作

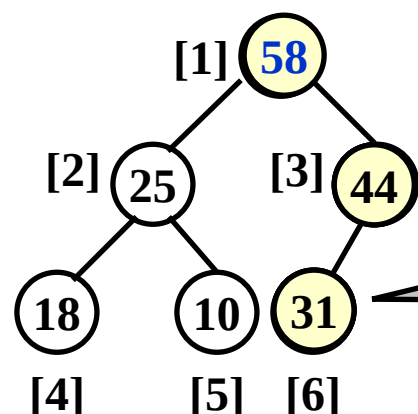
👉 最大堆的创建

```
typedef struct HeapStruct *MaxHeap;
struct HeapStruct {
    ElementType *Elements; /* 存储堆元素 */
    int Size; /* 堆的当前元素个数 */
    int Capacity; /* 堆的最大容量 */
};
```

把MaxData换成
小于堆中所有元素的
MinData，同样适用于
创建**最小堆**。

```
MaxHeap Create( int MaxSize )
{
    /* 创建容量为MaxSize的空的最大堆 */
    MaxHeap H = malloc( sizeof( struct HeapStruct ) );
    H->Elements = malloc( (MaxSize+1) * sizeof( ElementType ) );
    H->Size = 0;
    H->Capacity = MaxSize;
    H->Elements[0] = MaxData;
    /* 定义“哨兵”为大于堆中所有可能元素的值，便于以后更快操作 */
    return H;
}
```

👉 最大堆的插入



从堆的完全二叉树结构要求来说，
新结点的位置必须在这里。

Case 1 : new_item = 20

20 < 31 ✓

Case 2 : new_item = 35

35 > 31 35 < 44 ✓

Case 3 : new_item = 58

58 > 31 58 > 44 58 < MaxData ✓

❖ 算法可以概括成一句话：从新增的最后一个结点的父结点开始，用要插入元素向下过滤上层结点（相当于要插入的元素向上渗透）。

```
void Insert( MaxHeap H, ElementType item )
{ /* 将元素item 插入最大堆H，其中H->Element[0] 定义为哨兵 */
    int i;
    if ( IsFull(H) ) {
        printf("最大堆已满\n");
        return;
    }
    i = ++H->Size; /* i指向插入后堆中的最后一个元素的位置 */
    for ( ; H->Elements[i/2] < item; i/=2 )
        H->Elements[i] = H->Elements[i/2]; /* 向下过滤结点 */
    H->Elements[i] = item; /* 将item 插入 */
}
```

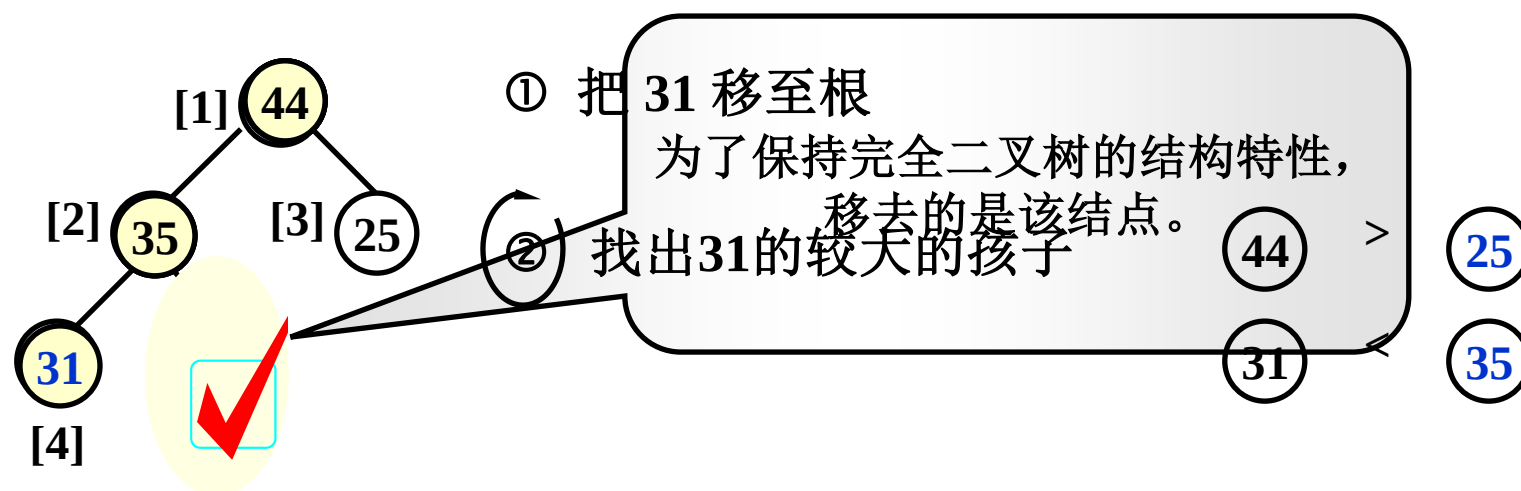
H->Element[0] 定义为哨兵
它不小于堆中任何元素
控制顺序
item 往上渗透

比交换数据要快

$T(N) = O(\log N)$

👉 最大堆的删除

➤ 取出根结点（最大值）元素，同时删除堆的一个结点。



$$T(N) = O(\log N)$$

```

ElementType DeleteMax( MaxHeap H )
{
    /* 从最大堆H中取出键值为最大的元素，并删除一个结点 */
    int Parent, Child;
    ElementType MaxItem, temp;
    if ( IsEmpty(H) ) {
        printf("最大堆已为空");
        return;
    }
    MaxItem = H->Elements[1];
    /* 取出根 */
    /* 用最大堆中最后一个元素从根结点开始向上 */
    temp = H->Elements[H->Size--];
    for( Parent=1; Parent*2 < H->Size; Parent=Child ) {
        Child = Parent*2;
        if( (Child!= H->Size) &&
            (H->Elements[Child] < H->Elements[Child+1]) )
            Child++;
        /* Child指向左右结点的较大者 */
        if( temp >= H->Elements[Child] ) break;
        else /* 移动temp元素到下一层 */
            H->Elements[Parent] = H->Elements[Child];
    }
    H->Elements[Parent] = temp;
    return MaxItem;
}

```

是否可以增加
一个哨兵而不用
该判断?

不用该条件
将会怎样?

新元素向下渗透

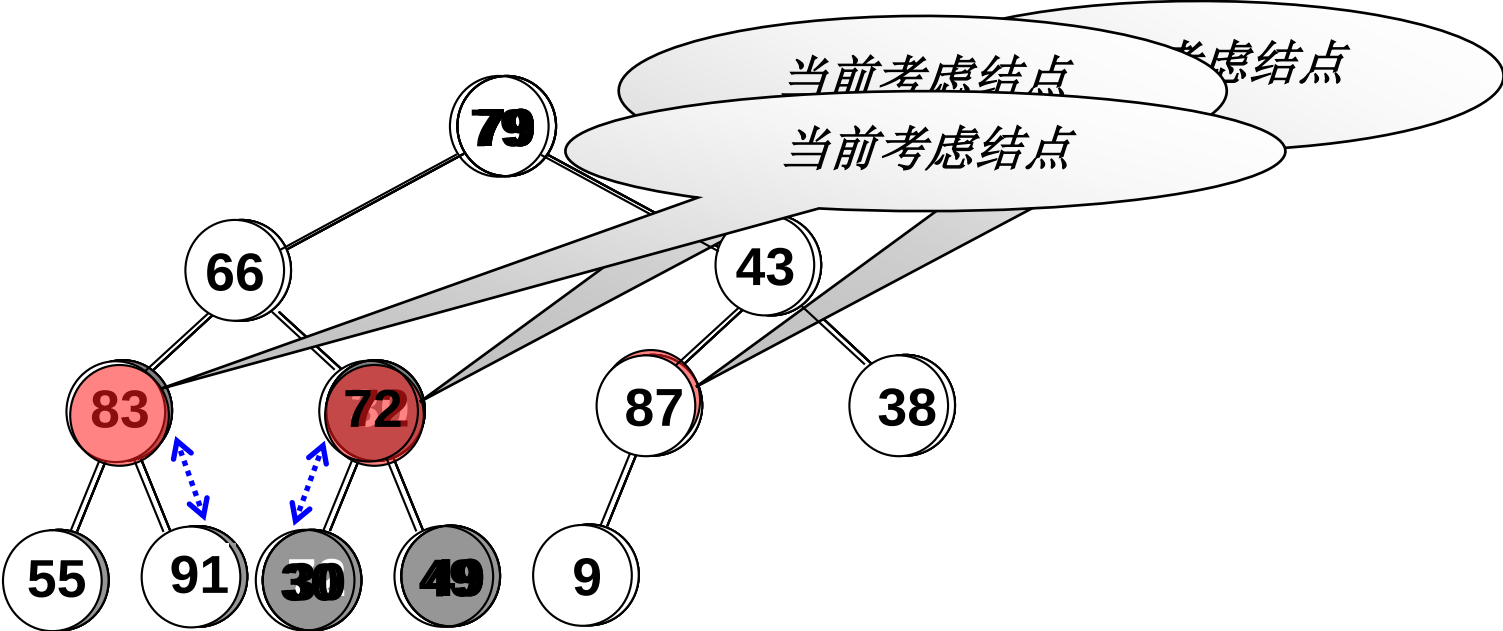
最大堆的建立

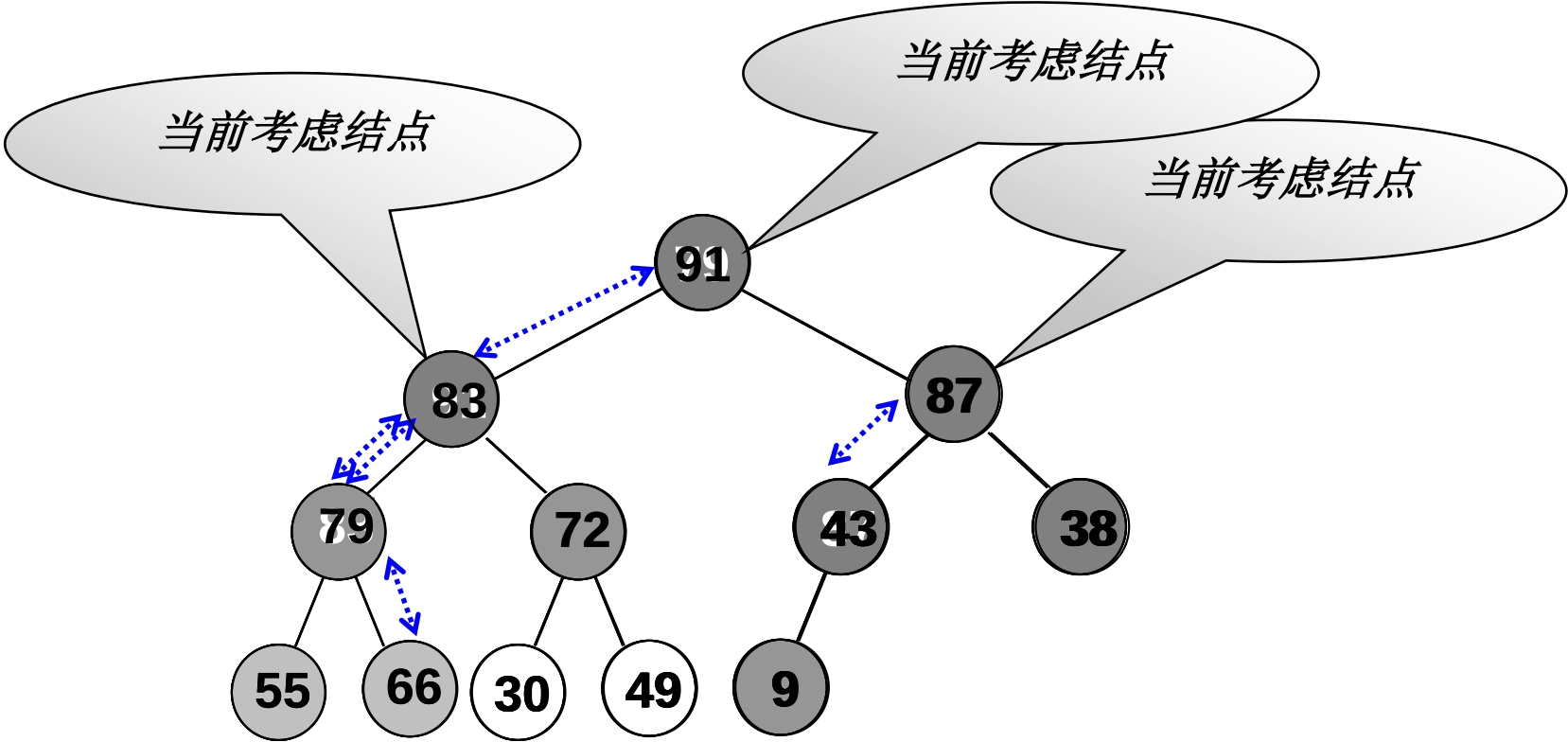
“建立最大堆”是指如何将已经存在的 N 个元素按最大堆的要求存放在一个一维数组中。

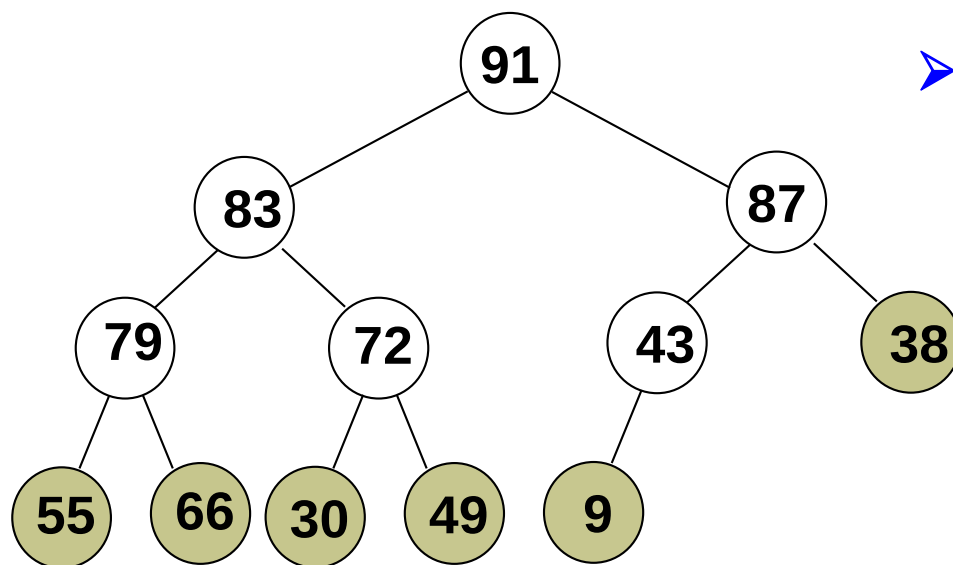
➤ 可以通过最大堆的插入操作，将 N 个元素一个个相继插入到一个初始为空的堆中去，其时间代价最大为 $O(N \log N)$ 。

➤ 在线性时间复杂度下建立最大堆。具体分两步进行：
第一步，将 N 个元素按输入顺序存入二叉树中，这一步只要求满足完全二叉树的结构特性，而不管其有序性。

第二步，调整各结点元素，以满足最大堆的有序特性。







➤ 线性时间复杂度 $T(n)=O(n)$

结点数	最多交换次数
$n/4$	1
$n/8$	2
$n/16$	3
.....	
$n/2^k=1$	$\log_2 n - 1$ (k-1)

$$T(n) = \frac{n}{4} + \frac{n}{8} \times 2 + \frac{n}{16} \times 3 + \dots + \frac{n}{2^k} \times (k-1)$$

$$2T(n) = \frac{n}{2} + \frac{n}{4} \times 2 + \frac{n}{8} \times 3 + \dots + \frac{n}{2^{k-1}} \times (k-1)$$

$$2T(n) - T(n) = \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + \frac{n}{2^{k-1}} - \frac{n}{2^k} \times (k-1) \leq n - (\log_2 n - 1) \leq n$$

```

MaxHeap BuildMaxHeap( MaxHeap H )
{
    /* 这里假设所有H->Size个元素已经存在H->Elements[]中 */
    /* 本函数将H->Elements[]中的元素调整，使满足最大堆的有序性 */
    int Parent, Child, i;
    ElementType temp;
    for( i = H->Size/2; i>0; i-- ){ /*从最后一个结点的父结点开始 */
        temp = H->Elements[i];
        for( Parent=i; Parent*2<=H->Size; Parent=Child ) {
            /* 向下过滤 */
            Child = Parent * 2;
            if( (Child!= H->Size) &&
                (H->Elements[Child] < H->Elements[Child+1]) )
                Child++; /* Child指向左右子结点的较大者 */
            if( temp >= H->Elements[Child] ) break;
            else /* 移动temp元素到下一层 */
                H->Elements[Parent] = H->Elements[Child];
        } /* 结束内部for循环对以H->Elements[i]为根的子树的调整 */
        H->Elements[Parent] = temp;
    }
    return H;
}

```

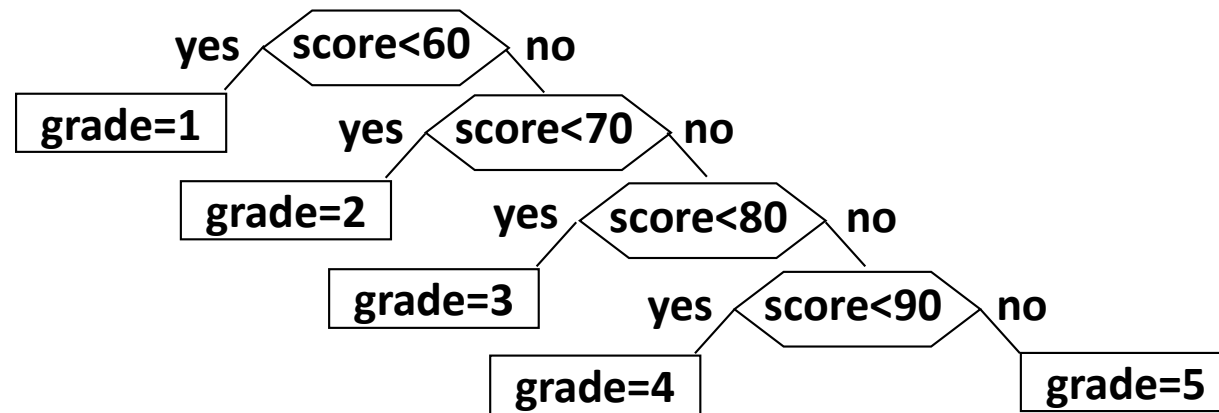
❖ 哈夫曼树 (Huffman Tree)

➤ 问题提出:

[例4.9] 写一个程序将百分制的考试成绩转换成五分制的成绩。

```
if( score < 60 ) grade =1;  
else if( score < 70 ) grade =2;  
else if( score < 80 ) grade =3;  
else if( score < 90 ) grade =4;  
else grade =5;
```

➤ 判定树:

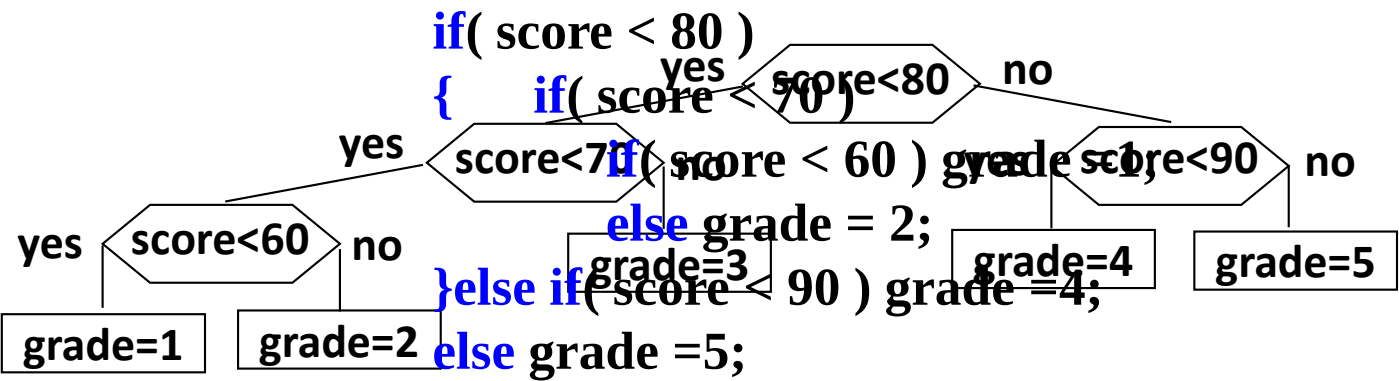


➤ 如果考虑学生成绩的分布概率：

分数段	0-59	60-69	70-79	80-89	90-100
比例	0.05	0.15	0.40	0.30	0.10

➤ 查找效率： $0.05 \times 1 + 0.15 \times 2 + 0.4 \times 3 + 0.3 \times 4 + 0.1 \times 4 = 3.15$

➤ 修改判定树：



➤ 查找效率： $0.05 \times 3 + 0.15 \times 3 + 0.4 \times 2 + 0.3 \times 2 + 0.1 \times 2 = 2.2$

❖ 哈夫曼树的定义

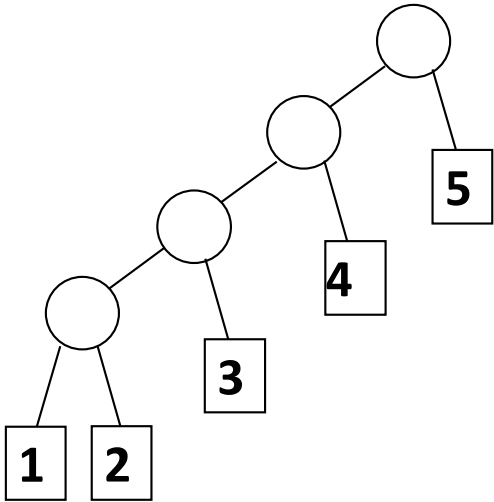
【定义】设一棵二叉树有 n 个叶子结点，每个叶子结点带有权值 w_k ，从根结点到每个叶子结点的长度为 l_k ，则每个叶子结点的带权路径长度之和就是这棵树的“带权路径长度（Weighted Path

Length，简称 **WPL**）”，即为：

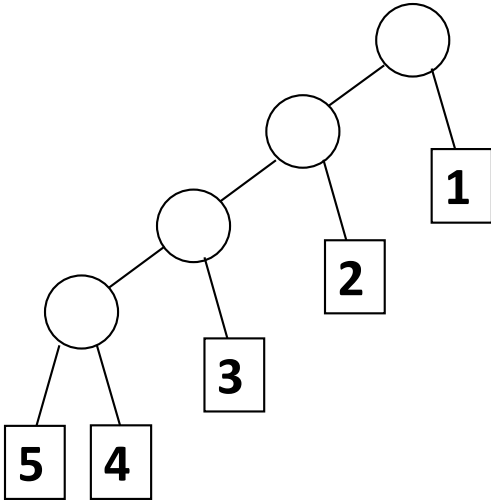
$$WPL = \sum_{k=1}^n w_k l_k \quad \circ$$

【定义】假设有 n 个权值 $\{w_1, w_2, \dots, w_n\}$ ，构造有 n 个叶子的二叉树，每个叶子的权值是 n 个权值之一。这样的二叉树也许可以构造多个，其中必有一个（或几个）是带权路径长度 **WPL** 最小的。达到 **WPL** 最小的二叉树就称为**最优二叉树**或**哈夫曼树**。

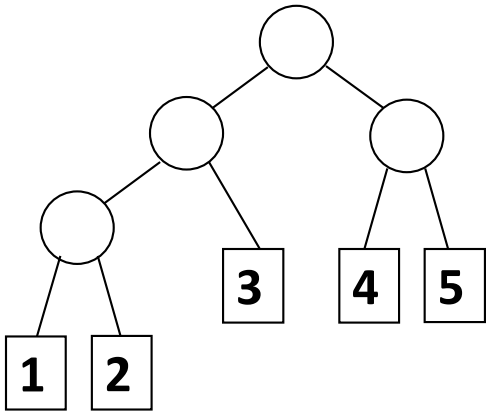
【例】有五个叶子结点，它们的权值为{1,2,3,4,5}，用此权值序列可以构造出形状不同的多个二叉树。



WPL = 34



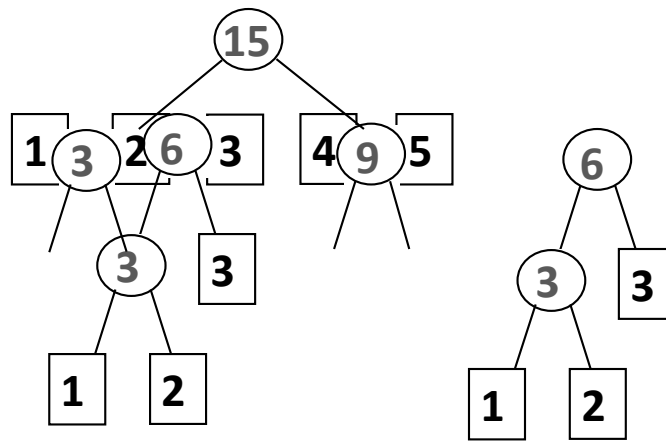
WPL = 50



WPL = 33

~~WPL = 1 × 4 + 2 × 3 + 3 × 3 + 4 × 4 + 5 × 4 = 53~~
WPL = 1 × 1 + 2 × 2 + 3 × 3 + 4 × 4 + 5 × 4 = 34

❖ 哈夫曼树的构造



👉 每次把**权值最小的两棵**二叉树合并

```

typedef struct TreeNode *HuffmanTree;
struct TreeNode{
    int Weight;
    HuffmanTree Left, Right;
}
HuffmanTree Huffman( MinHeap H )
{
    /* 假设H->Size个权值已经存在H->Elements[]->Weight里 */
    int i;    HuffmanTree T;
    BuildMinHeap(H); /*将H->Elements[]按权值调整为最小堆*/
    for (i = 1; i < H->Size; i++) { /*做H->Size-1次合并*/
        T = malloc( sizeof( struct TreeNode) ); /*建立新结点*/
        T->Left = DeleteMin(H);
            /*从最小堆中删除一个结点，作为新T的左子结点*/
        T->Right = DeleteMin(H);
            /*从最小堆中删除一个结点，作为新T的右子结点*/
        T->Weight = T->Left->Weight+T->Right->Weight;
            /*计算新权值*/
        Insert( H, T ); /*将新T插入最小堆*/
    }
    T = DeleteMin(H);
    return T;
}

```

整体复杂度为 $O(N \log N)$

❖ 哈夫曼树的特点:

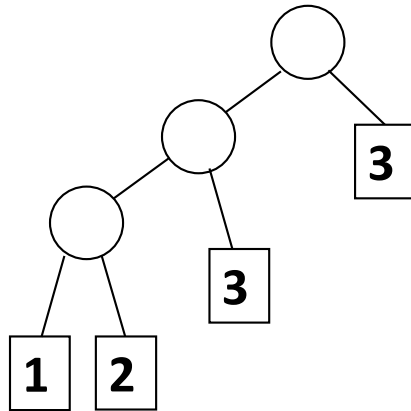
- 👉 没有度为1的结点;
- 👉 n 个叶子结点的哈夫曼树共有 $2n-1$ 个结点;
- 👉 哈夫曼树的任意非叶节点的左右子树交换后仍是哈夫曼树;

【定义】在不考虑结点权值的情况下，如果二叉树A通过任意结点的左右子树交换，可以变成二叉树B，那么就称A和B是同构的。

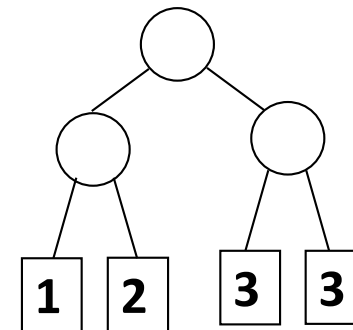
- 👉 也就是说，与一棵哈夫曼树同构的二叉树都是哈夫曼树;
- 👉 对同一组权值 $\{w_1, w_2, \dots, w_n\}$ ，是否存在不同构的两棵哈夫曼树呢？

Yes !

► 对一组权值{ 1, 2, 3, 3 }, 不同构的两棵哈夫曼树:



$$\text{WPL} = 18$$



$$\text{WPL} = 18$$

❖ 哈夫曼编码

- 给定一段字符串，如何对字符进行编码，可以使得该字符串的编码存储空间最少？
- 最优解决方案是“哈夫曼(Huffman)编码”。

[例4.10] 假设有一段文本，包含58个字符。经过统计，发现其中只有7个字符是互不相同的，它们是：a, e, i, s, t, 空格(sp)，换行(nl)。

【分析】如果用等长ASCII编码： $58 \times 8 = 464$ 位；
如果用等长3位编码： $58 \times 3 = 174$ 位；
如果出现频率高的字符用的编码短些，出现频率低的字符则可以编码长些；以期得到总的编码长度最短。

❖ 二叉树用于编码

➤ 如果字符 C_i 在深度 d_i 的地方并且出现的频率是 f_i , 那么编码总长

$$L = \sum d_i f_i$$

(00001001001011)

16

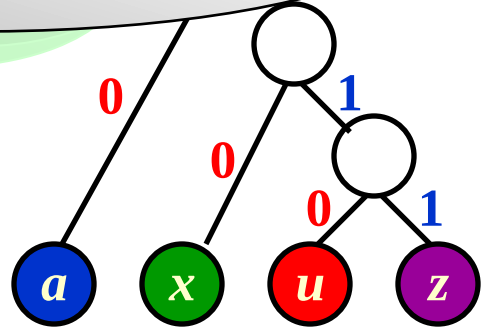
对如下编码方式,

$a = 0, u = 110, x = 10, z = 111$

码串 00010110

该如

所有结点要么有两个孩子, 要么是叶子。



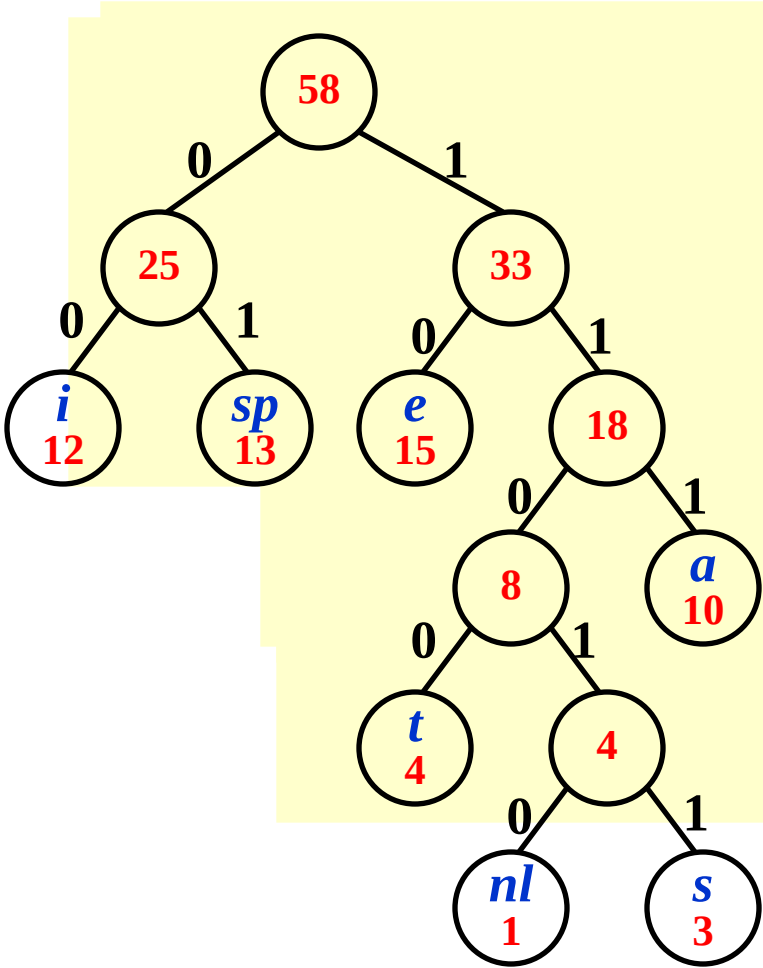
👁 所有字符都只出现在“满二叉树” (full tree) 的叶子上时任何码串都可以无二义地解码——这样的码叫做前缀码prefix code.



求编码总长最小的“满二叉树”，其中所有字符对应于二叉树的叶子结点。

【例】

C_i	a	e	i	s	t	sp	nl
f_i	10	15	12	3	4	13	1



$a : 111$
 $e : 10$
 $i : 00$
 $s : 11011$
 $t : 1100$
 $sp : 01$
 $nl : 11010$

$Cost = 3 \times 10 + 2 \times 15$
 $+ 2 \times 12 + 5 \times 3$
 $+ 4 \times 4 + 2 \times 13$
 $+ 5 \times 1$
 $= 146$

❖ 集合及其运算

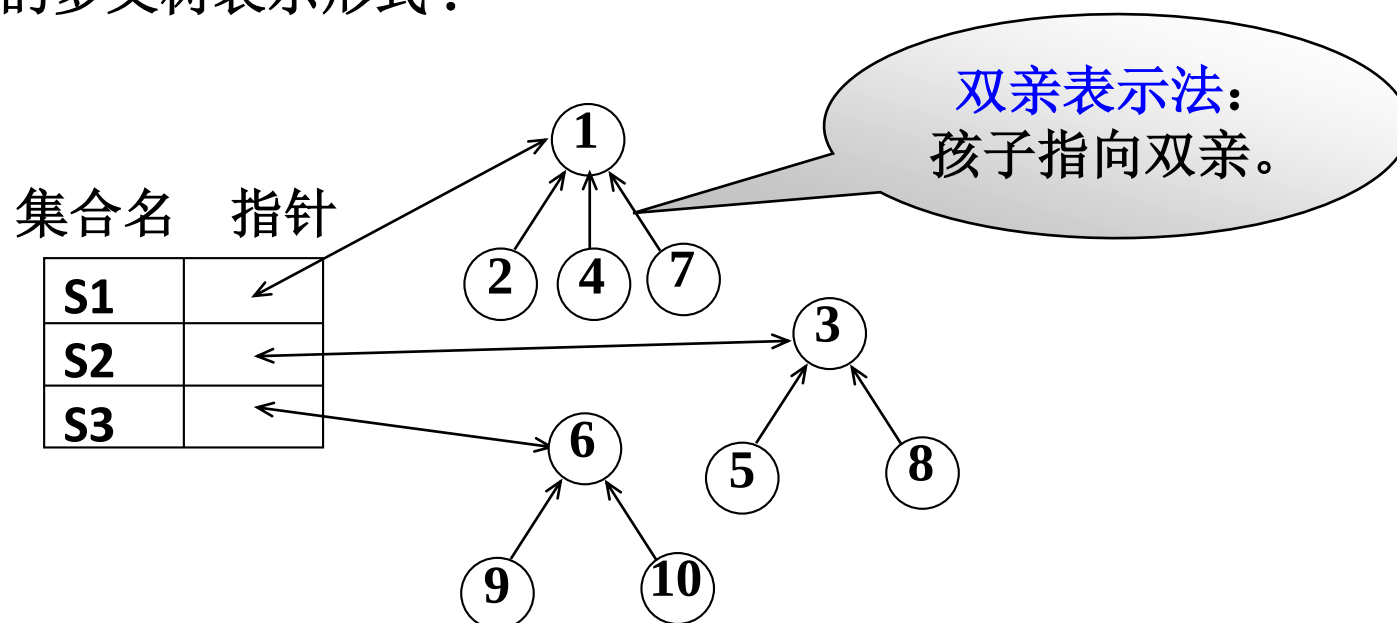
➤ 集合的表示

➤ 集合运算包括交、并、补、差以及判定一个元素是否是某一集合中等。

➤ 逻辑上，可以用树结构表示集合，树的每个结点代表一个集合元素。

➤ 例如，有三个互不相交的整数集合 $S1=\{1,2,4,7\}$ 、 $S2=\{3,5,8\}$ 、 $S3=\{6,9,10\}$ ，

➤ 这三个集合的多叉树表示形式：



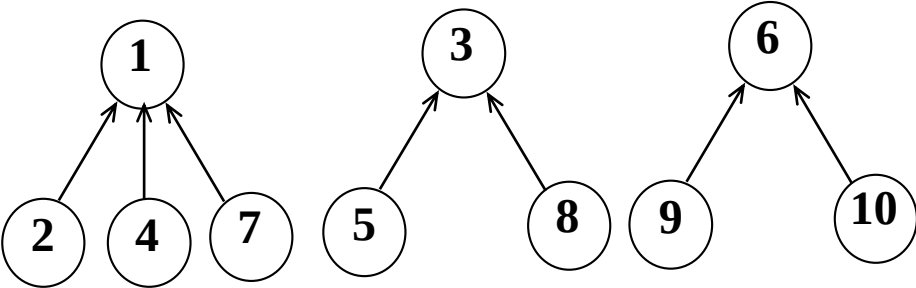
➤ 采用数组存储形式

负数表示根结点；
非负数表示双亲结点的下标。

下标	Data	Parent
0	1	-1
1	2	0
2	3	-1
3	4	0
4	5	2
5	6	-1
6	7	0
7	8	2
8	9	5
9	10	5

数组中每个元素的类型描述为：

```
typedef struct {  
    ElementType Data;  
    int Parent;  
} SetType;
```



► 集合运算

(1) 查找某个元素所在的集合（用根结点表示）

```
int Find( SetType S[ ], ElementType X )
{
    /* 在数组s中查找值为x的元素所属的集合 */
    /* MaxSize是全局变量，为数组s的最大长度 */
    int i;
    for ( i=0; i < MaxSize && S[i].Data != X; i++) ;
    if( i >= MaxSize ) return -1; /* 未找到x, 返回-1 */
    for( ; S[i].Parent > 0; i = S[i].Parent ) ;
    return i; /* 找到x所属集合，返回树根结点在数组s中的下标 */
}
```

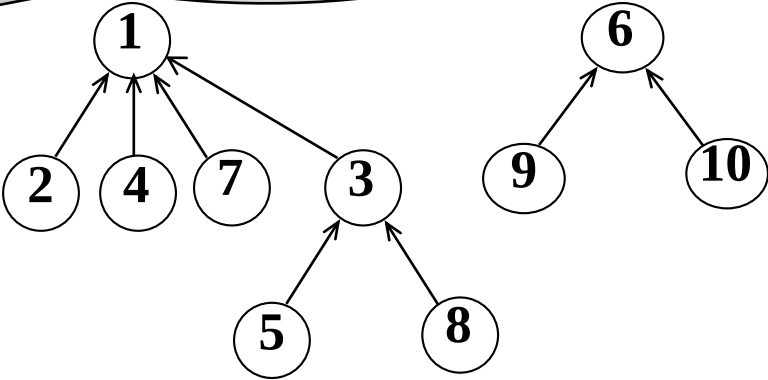
➤ 集合运算

(2) 集合的并运算

首先分别找到X1和X2两个元素所在集合树的根结点，如果它们不同根，则将其中一个根结点的父结点指针设置成另一个根结点的数组下标就行了。

```
void Union( SetType S[ ], ElementType X1, ElementType X2 )
{
    int Root1, Root2;
    Root1 = Find(S, X1);
    Root2 = Find(S, X2);
    if ( Root1 != Root2 ) S[Root2].Parent = Root1;
}
```

下标	data	Parent
0	1	-1
1	2	0
2	3	0
3	4	0
4	5	2
5	6	-1
6	7	0
7	8	2
8	9	5
9	10	5



改成-7。表示集合有7个元素

改成-3。表示集合有3个元素

为了改善合并以后的查找性能，
需要修改 Union 函数！
操作需，我们
可以，元素个数记录在
根结点中。